

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №4

«Запросы на выборку и модификацию данных. Представления.

Работа с индексами»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Тихонов Роман Александрович

Факультет Прикладной информатики

Группа К3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы:

Овладеть практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных и индексов.

Практическое задание:

1. Создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию лабораторной работы №2, часть 2 и 3).
2. Составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов.
3. Изучить графическое представление запросов и просмотреть историю запросов.
4. Создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с индексами. Для получения плана запроса использовать команду EXPLAIN.

Схема логической модели базы данных ERD:

<https://drive.google.com/file/d/1YzJLXw0hJ5flexoCZMJeb67hrcEzqjgc/view?usp=sharing>

Выполнение:

Наименование БД: Courses.

Вариант: 7.

1. Запросы к базе данных согласно индивидуальному заданию № 2:

Формулировка запроса № 1: Вывести все номера групп и программы, где количество слушателей меньше 10.

SQL-команда запроса № 1:

```
SET search_path TO lr3;
```

```

SELECT
    g.group_number,
    ep.program_name,
    COUNT(t.trainee_id) AS trainee_count
FROM training_group g
JOIN educational_program ep ON ep.program_id = g.program_id
LEFT JOIN trainee t ON t.group_id = g.group_id
GROUP BY g.group_id, g.group_number, ep.program_name
HAVING COUNT(t.trainee_id) < 10
ORDER BY trainee_count, g.group_number;

```

Результат выполнения запроса № 1:

	group_number character varying (10) 🔒	program_name character varying (200) 🔒	trainee_count bigint 🔒
1	ФР-23	Frontend-разработчик	0
2	ДС-24	Data Scientist	1
3	ФР-24	Frontend-разработчик	2

Формулировка запроса № 2: Вывести список преподавателей с указанием количества программ, где они преподавали за истекший учебный год.

SQL-команда запроса № 2:

```

SELECT
    tch.last_name,
    tch.first_name,
    COUNT(DISTINCT g.program_id) AS programs_count
FROM teacher tch
JOIN lesson l
    ON tch.teacher_id = l.teacher_id
JOIN training_group g
    ON l.group_id = g.group_id
WHERE l.lesson_date >= (CURRENT_DATE - INTERVAL '1 year')
GROUP BY
    tch.teacher_id,
    tch.last_name,
    tch.first_name;

```

Результат выполнения запроса № 2:

	last_name character varying (30) 🔒	first_name character varying (30) 🔒	programs_count bigint 🔒
1	Сидоров	Олег	1
2	Кузнецова	Елена	1

Формулировка запроса № 3: Вывести список преподавателей, которые не проводят занятия на третьей паре ни в один из дней недели.

SQL-команда запроса № 3:

```
SELECT
    teacher_id,
    last_name,
    first_name,
    middle_name
FROM teacher
WHERE teacher_id NOT IN (
    SELECT
        teacher_id
    FROM lesson
    WHERE lesson_time = '12:00:00'
)
ORDER BY last_name, first_name;
```

Результат выполнения запроса № 3:

	teacher_id [PK] integer ✎	last_name character varying (30) ✎	first_name character varying (30) ✎	middle_name character varying (30) ✎
1	3	Васильев	Петр	Алексеевич
2	2	Кузнецова	Елена	Павловна

Формулировка запроса № 4: Вывести список свободных лекционных аудиторий на ближайший понедельник.

SQL-команда запроса № 4:

```
SELECT
    r.room_number,
    r.capacity
FROM room r
```

```

JOIN room_type rt ON rt.room_type_id = r.room_type_id
WHERE rt.type_name = 'Лекционная'
  AND r.room_id NOT IN (
    SELECT
      room_id
    FROM lesson
    WHERE lesson_date = DATE_TRUNC('week', CURRENT_DATE + INTERVAL
'1 week')::DATE
  )
ORDER BY r.capacity DESC, r.room_number;

```

Результат выполнения запроса № 4:

	room_number character varying (6) 	capacity integer 
1	501	80
2	301	50

Формулировка запроса № 5: Вычислить общее количество обучающихся по каждой программе за последний год.

SQL-команда запроса № 5:

```

SELECT
  ep.program_name,
  COUNT(t.trainee_id) AS total_trainees
FROM educational_program ep
JOIN training_group g ON g.program_id = ep.program_id
LEFT JOIN trainee t ON t.group_id = g.group_id
WHERE g.start_date <= CURRENT_DATE
  AND g.end_date >= CURRENT_DATE - INTERVAL '1 year'
GROUP BY ep.program_id, ep.program_name
ORDER BY total_trainees DESC, ep.program_name;

```

Результат выполнения запроса № 5:

	program_name character varying (200) 🔒	total_trainees bigint 🔒
1	Frontend-разработчик	0

Формулировка запроса № 6: Вычислить среднюю загруженность компьютерных классов в неделю за последний месяц (в часах).

SQL-команда запроса № 6:

```
SELECT
    r.room_number,
    ROUND((COUNT(l.lesson_id) * 2.0 / 4.0), 2) AS
avg_weekly_load_hours
FROM room r
JOIN room_type rt ON rt.room_type_id = r.room_type_id
LEFT JOIN lesson l ON l.room_id = r.room_id
                AND l.lesson_date >= (CURRENT_DATE - INTERVAL '1
month')
WHERE rt.type_name = 'Компьютерный класс'
GROUP BY r.room_id, r.room_number
ORDER BY avg_weekly_load_hours DESC, r.room_number;
```

Результат выполнения запроса № 6:

	room_number character varying (6) 🔒	avg_weekly_load_hours numeric 🔒
1	405	0.50

Формулировка запроса № 7: Найти самые популярные программы за последние 3 года.

SQL-команда запроса № 7:

```
SELECT
    ep.program_name,
    COUNT(t.trainee_id) AS total_trainees
FROM educational_program ep
JOIN training_group g ON g.program_id = ep.program_id
JOIN trainee t ON t.group_id = g.group_id
WHERE g.end_date >= (CURRENT_DATE - INTERVAL '3 years')
```

```
GROUP BY ep.program_id, ep.program_name
ORDER BY total_trainees DESC, ep.program_name;
```

Результат выполнения запроса № 7:

	program_name character varying (200) 🔒	total_trainees bigint 🔒
1	Frontend-разработчик	2
2	Data Scientist	1

2. Запросы на создание представлений согласно индивидуальному заданию, ч. 3:

Формулировка запроса на создание представления № 1: Создать представление для потенциальных слушателей, содержащее перечень специальностей, изучаемых на них дисциплин и количество часов.

SQL-команда запроса на создание представления № 1:

```
CREATE OR REPLACE VIEW vw_potential_trainees AS
SELECT
    ep.program_name AS "Специальность",
    d.discipline_name AS "Дисциплина",
    (d.lecture_hours + d.practice_hours + d.lab_hours) AS "Часов по дисциплине",
    ep.total_hours AS "Общее количество часов (программа)"
FROM educational_program ep
JOIN program_discipline pd ON pd.program_id = ep.program_id
JOIN discipline d ON d.discipline_id = pd.discipline_id;
```

SQL-команда для просмотра содержимого представления № 1:

```
SELECT * FROM vw_potential_trainees;
```

Содержимое представления № 1:

	Специальность character varying (200) 🔒	Дисциплина character varying (200) 🔒	Часов по дисциплине integer 🔒	Общее количество часов (программа) integer 🔒
1	Frontend-разработчик	Веб-разработка	50	120
2	Data Scientist	Машинное обучение	50	350
3	Data Scientist	Веб-разработка	50	350

Формулировка запроса на создание представления № 2: Создать представление общих доход по каждой программе за последний год.

SQL-команда запроса на создание представления № 2:

```
CREATE OR REPLACE VIEW vw_program_revenue AS
SELECT
    ep.program_name AS "Программа",
    COUNT(t.trainee_id) AS "Количество слушателей",
    (COUNT(t.trainee_id) * ep.price) AS "Общий доход"
FROM educational_program ep
JOIN training_group g ON g.program_id = ep.program_id
JOIN trainee t ON t.group_id = g.group_id
WHERE g.start_date >= (CURRENT_DATE - INTERVAL '1 year')
GROUP BY ep.program_id, ep.program_name, ep.price;
```

SQL-команда для просмотра содержимого представления № 2:

```
SELECT * FROM vw_program_revenue;
```

Содержимое представления № 2:

	Программа character varying (200)	Количество слушателей bigint	Общий доход numeric
1	Frontend-разработчик	2	120000.00

3. Запросы на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов:

Формулировка запроса UPDATE с подзапросом: Прибавить 10 лекторских часов в каждую дисциплину, которая входит в программу “Data Scientist”.

SQL-команда для просмотра количества лекторских часов у дисциплин, которые входят в программу “Data Scientist”:

```
SELECT
    d.discipline_name,
    d.lecture_hours,
    ep.program_name
FROM discipline d
JOIN program_discipline pd ON pd.discipline_id = d.discipline_id
```



```
JOIN educational_program ep ON ep.program_id = pd.program_id
WHERE ep.program_name = 'Data Scientist'
ORDER BY d.discipline_name;
```

Результат до запроса UPDATE:

	discipline_name character varying (200) 🔒	lecture_hours integer 🔒	program_name character varying (200) 🔒
1	Веб-разработка	10	Data Scientist
2	Машинное обучение	20	Data Scientist

SQL-команда запроса UPDATE:

```
UPDATE discipline
SET lecture_hours = lecture_hours + 10
WHERE discipline_id IN (
    SELECT
        pd.discipline_id
    FROM program_discipline pd
    JOIN educational_program ep ON ep.program_id = pd.program_id
    WHERE ep.program_name = 'Data Scientist'
);
```

Результат после выполнения запроса UPDATE:

	discipline_name character varying (200) 🔒	lecture_hours integer 🔒	program_name character varying (200) 🔒
1	Веб-разработка	20	Data Scientist
2	Машинное обучение	30	Data Scientist

Формулировка запроса INSERT с подзапросом: Добавить документ конкретному слушателю с заданной почтой.

SQL-команда для просмотра документов конкретного слушателя:

```
SELECT
    t.trainee_id,
    t.last_name,
    t.first_name,
    td.document_id,
    td.document_number,
```

```

        td.issue_date,
        dt.type_name AS document_type
FROM trainee t
LEFT JOIN trainee_document td
    ON td.trainee_id = t.trainee_id
LEFT JOIN document_type dt
    ON dt.document_type_id = td.document_type_id
WHERE t.email = 'novikova@test.com'
ORDER BY
    td.issue_date,
    td.document_id;

```

Результат до запроса INSERT:

	trainee_id integer	last_name character varying (30)	first_name character varying (30)	document_id integer	document_number character varying (50)	issue_date date	document_type character varying (50)
1	2	Новикова	Ольга	[null]	[null]	[null]	[null]

SQL-команда запроса INSERT:

```

INSERT INTO trainee_document (
    trainee_id,
    document_number,
    issue_date,
    document_type_id
)
VALUES (
    (SELECT trainee_id FROM trainee WHERE email =
'novikova@test.com'),
    'УДОСТ-001',
    DATE '2024-06-15',
    (SELECT document_type_id FROM document_type WHERE type_name =
'Удостоверение')
);

```

Результат после выполнения запроса INSERT:

	trainee_id integer	last_name character varying (30)	first_name character varying (30)	document_id integer	document_number character varying (50)	issue_date date	document_type character varying (50)
1	2	Новикова	Ольга	3	УДОСТ-001	2024-06-15	Удостоверение

Формулировка запроса DELETE с подзапросом: Удалить документы слушателей, которые были отчислены.

SQL-команда для просмотра всех слушателей с их статусом обучения и документом, если есть:

```
SELECT
    t.last_name,
    t.first_name,
    ts.status_name,
    td.document_number
FROM trainee t
JOIN trainee_status ts ON ts.trainee_status_id = t.trainee_status_id
LEFT JOIN trainee_document td ON td.trainee_id = t.trainee_id
ORDER BY t.last_name;
```

Результат до запроса DELETE:

	last_name character varying (30)	first_name character varying (30)	status_name character varying (20)	document_number character varying (50)
1	Васильев	Николай	Учится	[null]
2	Михайлов	Денис	Учится	СВИД-999
3	Новикова	Ольга	Учится	УДОСТ-001
4	Смирнов	Алексей	Отчислен	СВИД-456
5	Тихонов	Роман	Учится	[null]

SQL-команда запроса DELETE:

```
DELETE FROM trainee_document
WHERE trainee_id IN (
    SELECT
        t.trainee_id
    FROM trainee t
    JOIN trainee_status ts ON ts.trainee_status_id =
t.trainee_status_id
    WHERE ts.status_name = 'Отчислен'
);
```

Результат после выполнения запроса DELETE:

	last_name character varying (30)	first_name character varying (30)	status_name character varying (20)	document_number character varying (50)
1	Васильев	Николай	Учится	[null]
2	Михайлов	Денис	Учится	СВИД-999
3	Новикова	Ольга	Учится	УДОСТ-001
4	Смирнов	Алексей	Отчислен	[null]
5	Тихонов	Роман	Учится	[null]

4. Создание индексов:

Формулировка запроса № 1 с простым индексом: Вывести список преподавателей, которые не проводят занятия на третьей паре ни в один из дней недели.

SQL-команда запроса № 1:

```
EXPLAIN ANALYZE
SELECT
    teacher_id,
    last_name,
    first_name,
    middle_name
FROM teacher
WHERE teacher_id NOT IN (
    SELECT
        teacher_id
    FROM lesson
    WHERE lesson_time = '12:00:00'
)
ORDER BY last_name, first_name;
```

Результат до создания индекса:

	QUERY PLAN	
	text	
1	Sort (cost=37.66..37.75 rows=35 width=238) (actual time=0.032..0.032 rows=2.00 loops=1)	
2	Sort Key: teacher.last_name, teacher.first_name	
3	Sort Method: quicksort Memory: 25kB	
4	Buffers: shared hit=2	
5	-> Seq Scan on teacher (cost=25.89..36.77 rows=35 width=238) (actual time=0.021..0.022 rows=2.00 lo...	
6	Filter: (NOT (ANY (teacher_id = (hashed SubPlan 1).col1)))	
7	Rows Removed by Filter: 1	
8	Buffers: shared hit=2	
9	SubPlan 1	
10	-> Seq Scan on lesson (cost=0.00..25.88 rows=6 width=4) (actual time=0.004..0.004 rows=1.00 loo...	
11	Filter: (lesson_time = '12:00:00':time without time zone)	
12	Rows Removed by Filter: 5	
13	Buffers: shared hit=1	
14	Planning Time: 0.092 ms	
15	Execution Time: 0.051 ms	

SQL-команда для создания индекса:

```
CREATE INDEX idx_lesson_time ON lesson(lesson_time);
```

Результат после создания индекса:

	QUERY PLAN
	text 
1	Sort (cost=19.93..20.01 rows=35 width=238) (actual time=0.056..0.057 rows=2.00 loops=1)
2	Sort Key: teacher.last_name, teacher.first_name
3	Sort Method: quicksort Memory: 25kB
4	Buffers: shared hit=2 read=1
5	-> Seq Scan on teacher (cost=8.15..19.03 rows=35 width=238) (actual time=0.041..0.042 rows=2.00 loops=1)
6	Disabled: true
7	Filter: (NOT (ANY (teacher_id = (hashed SubPlan 1).col1)))
8	Rows Removed by Filter: 1
9	Buffers: shared hit=2 read=1
10	SubPlan 1
11	-> Index Scan using idx_lesson_time on lesson (cost=0.13..8.15 rows=1 width=4) (actual time=0.021..0.022 rows=1.00 loo...
12	Index Cond: (lesson_time = '12:00:00':time without time zone)
13	Index Searches: 1
14	Buffers: shared hit=1 read=1
15	Planning:
16	Buffers: shared hit=15 read=1
17	Planning Time: 0.277 ms
18	Execution Time: 0.084 ms

Из скриншотов видно, что время примерно одинаковое, при использовании индексов время даже немного больше т.к. в БД мало данных. Таким образом, при малом количестве данных выгодно использовать поиск по строкам.

Формулировка запроса № 2 с составным индексом: Вывести список свободных лекционных аудиторий на ближайший понедельник.

SQL-команда запроса № 2:

```
EXPLAIN ANALYZE
WITH next_monday AS (
  SELECT
    (
      CURRENT_DATE
      + ((8 - EXTRACT(DOW FROM CURRENT_DATE)::int) % 7)
    )::date AS dt
)
SELECT
  r.room_number,
```

```

        r.capacity
FROM room r
JOIN room_type rt
    ON rt.room_type_id = r.room_type_id
CROSS JOIN next_monday nm
WHERE rt.type_name = 'Лекционная'
    AND NOT EXISTS (
        SELECT
            1
        FROM lesson l
        WHERE l.room_id = r.room_id
            AND l.lesson_date = nm.dt
    );

```

Результат до создания индекса:

	QUERY PLAN	
	text	
1	Nested Loop Anti Join (cost=8.18..34.53 rows=1 width=32) (actual time=0.067..0.073 rows=2.00 loops=1)	
2	Join Filter: (l.room_id = r.room_id)	
3	Buffers: shared hit=5	
4	-> Hash Join (cost=8.18..33.34 rows=1 width=36) (actual time=0.056..0.058 rows=2.00 loops=1)	
5	Hash Cond: (r.room_type_id = rt.room_type_id)	
6	Buffers: shared hit=3	
7	-> Seq Scan on room r (cost=0.00..22.00 rows=1200 width=40) (actual time=0.020..0.021 rows=3.00 loops=1)	
8	Buffers: shared hit=1	
9	-> Hash (cost=8.17..8.17 rows=1 width=4) (actual time=0.029..0.029 rows=1.00 loops=1)	
10	Buckets: 1024 Batches: 1 Memory Usage: 9kB	
11	Buffers: shared hit=2	
12	-> Index Scan using room_type_type_name_key on room_type rt (cost=0.15..8.17 rows=1 width=4) (actual time=0.025..0.026 rows=1.00 loo...	
13	Index Cond: ((type_name)::text = 'Лекционная'::text)	
14	Index Searches: 1	
15	Buffers: shared hit=2	
16	-> Seq Scan on lesson l (cost=0.00..1.18 rows=1 width=4) (actual time=0.006..0.006 rows=0.00 loops=2)	
17	Filter: (lesson_date = (CURRENT_DATE + ((8 - (EXTRACT(dow FROM CURRENT_DATE))::integer) % 7)))	
18	Rows Removed by Filter: 6	
19	Buffers: shared hit=2	
20	Planning Time: 0.265 ms	Активаци
21	Execution Time: 0.105 ms	Чтобы актив "Параметры

SQL-команда для создания индекса:

```
CREATE INDEX idx_lesson_room_date ON lesson(room_id, lesson_date);
```

Результат после создания составного индекса:

QUERY PLAN	
1	Nested Loop Anti Join (cost=8.48..77.70 rows=1 width=32) (actual time=0.044..0.048 rows=2.00 loops=1)
2	Buffers: shared hit=6
3	-> Hash Join (cost=8.33..77.49 rows=1 width=36) (actual time=0.039..0.041 rows=2.00 loops=1)
4	Hash Cond: (r.room_type_id = rt.room_type_id)
5	Buffers: shared hit=4
6	-> Index Scan using room_pkey on room r (cost=0.15..66.15 rows=1200 width=40) (actual time=0.019..0.020 rows=3.00 loops=1)
7	Index Searches: 1
8	Buffers: shared hit=2
9	-> Hash (cost=8.17..8.17 rows=1 width=4) (actual time=0.016..0.016 rows=1.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 9kB
11	Buffers: shared hit=2
12	-> Index Scan using room_type_type_name_key on room_type rt (cost=0.15..8.17 rows=1 width=4) (actual time=0.014..0.014 rows=1.00 loops=1)
13	Index Cond: ((type_name)::text = 'Лекционная'::text)
14	Index Searches: 1
15	Buffers: shared hit=2
16	-> Index Only Scan using idx_lesson_room_date on lesson l (cost=0.15..0.18 rows=1 width=4) (actual time=0.002..0.002 rows=0.00 loops=2)
17	Index Cond: ((room_id = r.room_id) AND (lesson_date = (CURRENT_DATE + ((8 - (EXTRACT(dow FROM CURRENT_DATE))::integer) % 7))))
18	Heap Fetches: 0
19	Index Searches: 2
20	Buffers: shared hit=2
21	Planning Time: 0.205 ms
22	Execution Time: 0.069 ms

Из скриншотов видно, что время при использовании составного индекса уменьшилось. Следовательно, использование составного индекса даже на малом объеме данных может ускорить выполнение запросов.

SQL-команда для удаления индексов:

```
DROP INDEX idx_lesson_time;
DROP INDEX idx_lesson_room_date;
```

Выводы:

В ходе лабораторной работы были получены практические навыки составления запросов на выборку и модификацию данных в PostgreSQL и работы с инструментами pgAdmin 4. Были разработаны и выполнены запросы на выборку с использованием соединений таблиц (JOIN), группировок (GROUP BY, HAVING), подзапросов и конструкций EXISTS/NOT EXISTS, что позволило получать сводную и аналитическую информацию по учебным группам, образовательным программам, преподавателям и аудиториям. Также были созданы представления (VIEW), упрощающие повторное использование типовых выборок: представление с перечнем программ и изучаемых на них дисциплин с

количеством часов, а также представление для расчёта условного дохода по программам за последний год согласно выбранной формуле. Для модификации данных были составлены запросы INSERT, UPDATE и DELETE с подзапросами, а корректность выполнения операций была подтверждена сравнением результатов выборок до и после изменения данных. Дополнительно была изучена работа с планами выполнения запросов с помощью EXPLAIN ANALYZE, рассмотрены основные элементы плана и показатели времени выполнения. Для двух запросов была выполнена индексация: в одном случае создан простой индекс, во втором – составной, после чего проведено сравнение планов и времени выполнения запросов до и после создания индексов. Таким образом, цель работы достигнута в полном объеме.